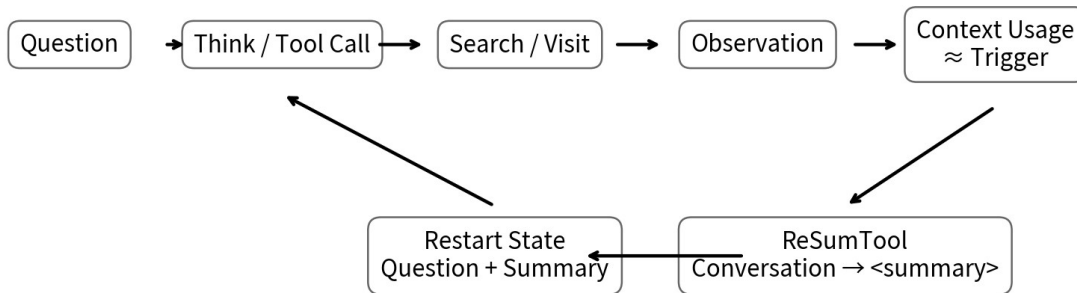


ReSum

Unlocking Long-Horizon Search Intelligence via Context Summarization

Deep Research 长程上下文管理：完整研究 • Know-Why / Know-How



不是“把摘要附加到旧历史”，而是把旧工作区整体替换为可重启的压缩状态。

研究主线：把 Agent 历史从“不可逆增长日志”转换为“可重启压缩状态”

研究日期：2026-08-13 | 论文版本：arXiv v3 (2026-03-26)

执行摘要

ReSum 研究的是一个非常基础但长期被低估的问题：Deep Research Agent 的失败往往不是“不会搜”，而是“搜得越久，工作记忆越糟”。ReAct 把 Thought / Action / Observation 全部线性追加到 Context，复杂任务需要更多搜索轮次时，Agent 会同时遭遇 token 上限、历史噪声、注意力稀释和旧证据难以定位。ReSum 的解决方式不是扩大模型 Context，也不是修改 Transformer，而是定期调用外部 Summary Tool，把长历史压缩成 goal-oriented summary，然后用 Question + Summary 重新启动一段新的工作上下文。[S1]

核心公式 $\text{ReSum} \approx \text{ReAct Policy} \times \text{Periodic Context Compression} \times \text{Restartable State} \times \text{Summary-conditioned Reasoning}$



核心演进：Context Window 从“被动容器”变成 Agent Runtime 中可管理、可训练、可验证的状态。

图 1 | Agent Context Management 的演进位置

最重要的 8 个结论

- Context Window 不是历史数据库，而是每一步推理的有限工作内存；把所有历史永久保留是一种架构选择，不是必然。
- ReSum 的关键动作不是“摘要”，而是“摘要后 Reset”：旧工作区被丢弃，只保留 Question + Summary。
- 摘要工具需要具备 evidence synthesis 能力，而不只是普通聊天摘要能力；专项训练的小模型可以胜过更大的通用模型。
- 训练-free ReSum 平均比 ReAct 提升约 4.5 个百分点，说明 Context Runtime 本身就能提升已有 Agent。
- ReSum-GRPO 把长轨迹按摘要点切成 segments，再把最终 trajectory advantage 广播给每段，从而解决 summary-conditioned reasoning 的训练适配。
- 只做 Recent History 截断通常不够：它释放 token，但破坏历史连续性；好的压缩必须保留关键证据与已完成推理。
- Rule-based summary trigger 是工程上的保守选择：稳定、可预测、便于预算，而“让模型自己决定何时压缩”仍可能不可靠。
- 2026 的后续工作已经把固定 summarization 推进到更丰富的 Context Action Space 与外部 Context Manager，ReSum 是这条演进线的重要基线。

1. 时代背景：为什么 32K / 128K 仍然不等于“长程 Agent 已解决”

ReSum 论文从 BrowseComp 等困难信息搜索任务出发：复杂问题包含多实体、交叉关系和不完整线索，必须经过连续的 targeted search、page visit、cross-verification 才能拼出完整 evidence chain。论文对 WebSailor-7B 的分析发现，正确样本通常能在 32K 内完成，而失败样本更常触及或超过 32K，说明 context exhaustion 与长程失败高度相关。[S1]

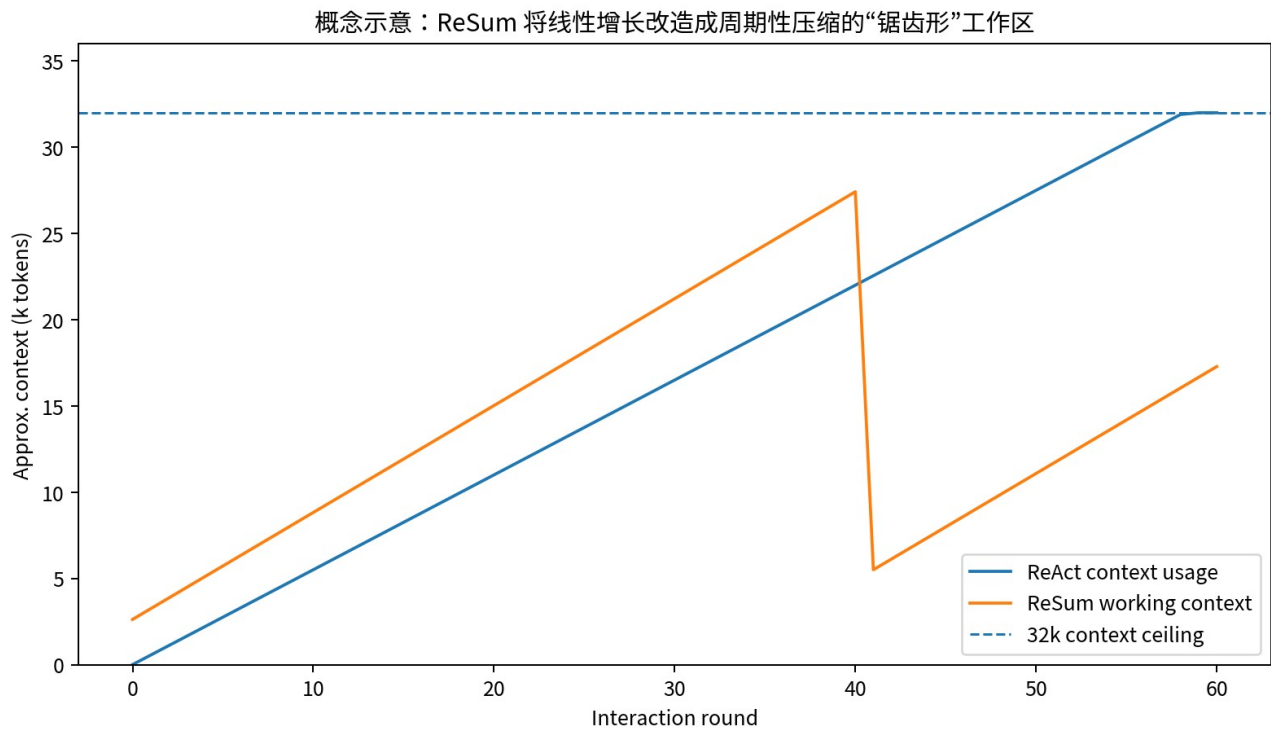


图 2 | 概念示意：ReAct 线性增长 vs ReSum 周期压缩

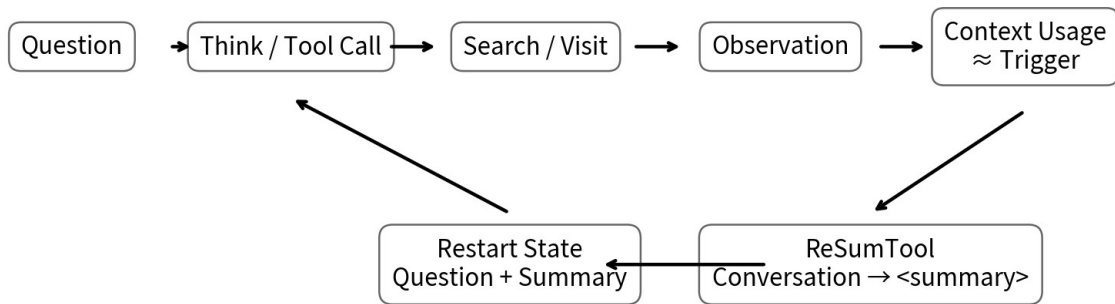
1.1 长 Context 的三个不同问题

问题	症状	仅扩大 Context 是否解决
硬上限	历史 token 超过窗口，任务被截断	部分解决，但任务 horizon 仍可继续增长
注意力稀释	关键线索埋在大量旧 Observation 中	不一定；更多 token 可能加重噪声
推理状态漂移	Agent 忘记已排除候选、重复搜索	不一定；需要结构化状态
成本/延迟	每一步重复处理更长前缀	更长窗口通常更贵

Know-Why Long-horizon Agent 的瓶颈不是“模型最多能读多少”，而是“Runtime 每一轮应该把哪些信息继续留在工作区”。

2. ReSum Paradigm：从 Append-only History 到 Restartable State

ReSum 前几轮与 ReAct 相同：Policy 根据当前历史生成 <think> 和 <tool_call>，Runtime 执行 Search / Visit 等工具，并把 <tool_response> 追加到 history。当压缩触发器激活时，Summary Tool 对整个积累历史生成 <summary>；随后 Runtime 构造 $q'=(original\ question, summary)$ ，并将工作历史直接 reset 为 q' 。之后 Agent 从这个新状态继续搜索。[S1]



不是“把摘要附加到旧历史”，而是把旧工作区整体替换为可重启的压缩状态。

图 3 | ReSum 的核心控制流

2.1 为什么“Reset”比“Append Summary”更重要

- 如果只是把 summary 附在旧 history 后面，token 仍持续增长。
- 旧网页片段、失败分支、重复查询仍会参与后续 attention，摘要无法真正去噪。
- Reset 把 summary 提升为新的 state representation：它决定下一阶段 Agent 能看到什么。
- 因此 ReSum 更像 checkpoint / state compaction，而不是聊天记录摘要。

2.2 Trigger：系统触发还是 Agent 自主触发？

论文明确选择 systematic trigger：当 context 接近上限时由 Runtime 触发 summary，而不是要求 Agent 自己判断。原因是自主管理上下文需要额外元认知能力，当前 Agent 未必稳定。官方源码还支持按固定 summary_iteration 周期触发；当 RESUM=True 且 token_count 接近 MAX_CONTEXT 的 90% 时也会触发。[S1] [S3]

生产建议 先用 deterministic trigger (token watermark / turn budget / tool-output spike)；只有在有专门 eval 后，才把“何时压缩”交给 Agent Policy。

3. Summary Tool 不是普通摘要器：它是 Search State Compiler

论文强调 ReSum 的 Summary Tool 需要对长且噪声很高的 web interaction history 做逻辑推理，提炼可验证 evidence，并保持对原问题的 goal consistency。通用 instruct 模型往往会生成碎片化摘要；reasoning model 更擅长按候选、证据、来源组织信息。[S1]



训练目标不是“写得短”，而是保留：关键证据 / 来源 / 已排除候选 / 当前结论 / 可继续推理的信息结构。

论文参数：batch 64, 2 epochs, learning rate 7e-6。

图 4 | ReSumTool-30B 的数据与训练管线

3.1 数据构造

- Explorer: WebSailor-30B 负责真实搜索与网页访问。
- Teacher Summarizer: GPT-OSS-120B 在压缩点生成高质量 summary。
- Task Source: SailorFog-QA, 选择原因是困难度高、真正需要多轮搜索。
- 数据规模: 超过 9K 的 <Conversation, Summary> / <Input, Output> pairs。
- Student: Qwen3-30B-A3B-Thinking, SFT 2 epochs, batch 64, lr 7e-6。

3.2 Summary 应该保留什么？

信息类型	为什么必须保留	建议表示
已确认事实	后续推理的硬约束	claim + source + confidence
候选实体	长链搜索经常需要逐一排除	candidate + matched/missing attributes
负证据/已排除项	防止重复搜索	candidate + rejection reason
未完成线索	决定下一轮 search query	open questions / unresolved links
来源身份	支撑最终可验证性	URL / title / source label
当前阶段结论	帮助新 segment 快速恢复方向	working hypothesis

有趣的是，论文在正式 summary prompt 中刻意不强制模型显式列出 information gaps 或 action plan，担心分散 summarization 主任务或陷入反复自检循环；作者观察到强 summary model 会自然保留部分 gap/next-step 信息。[S1]

4. 官方源码：ReSum 只改了 Agent Loop 的少数位置

Alibaba-NLP/DeepResearch 当前仓库在 WebAgent/WebResummer 中提供独立实现。react_agent.py 仍然是 MultiTurnReactAgent: 最多 MAX_LLM_CALL_PER_RUN（默认 60），维护 messages 与 full_trajectory，并在工具调用后更新历史。新增逻辑集中在 token counting、should_summarize、summary server 调用以及 messages reset。[S2][S3]

4.1 源码级关键逻辑

- MAX_CONTEXT 默认 32K；max_tokens = MAX_CONTEXT*1024 - 1000，为最后输出预留空间。
- should_summarize = (ReSum 且 token_count 达到阈值约 90%) 或 round % summary_iteration == 0。
- summarize_conversation(question, recent_messages, last_summary) 支持把上一轮 summary 递归注入下一次 summary prompt。
- 成功后 messages 被重建为 system + user(new_observation)，new_observation 包含原问题与 <summary>。
- full_trajectory 不丢失，因此评测/训练/调试仍能看到完整过程；工作 context 与审计日志被分离。
- Summary Server 采用 OpenAI-compatible 请求，最多 10 次 retry，timeout 360 秒，max_tokens 4096。

关键工程抽象 Working Context ≠ Full Trajectory Log。前者可被压缩/重写，后者应完整持久化用于审计、训练、复现和 failure analysis。

5. Training-free 实验：Runtime 改动本身有多大价值

论文在 GAIA、BrowseComp-ZH、BrowseComp 上比较 ReAct、Recent History、MEM1 与 ReSum，最大 tool calls 统一为 60，WebSailor-3B / 7B / 30B 均限制 32K context。整体上 ReSum 相对 ReAct 的平均绝对增益约 4.5 个百分点。[S1]

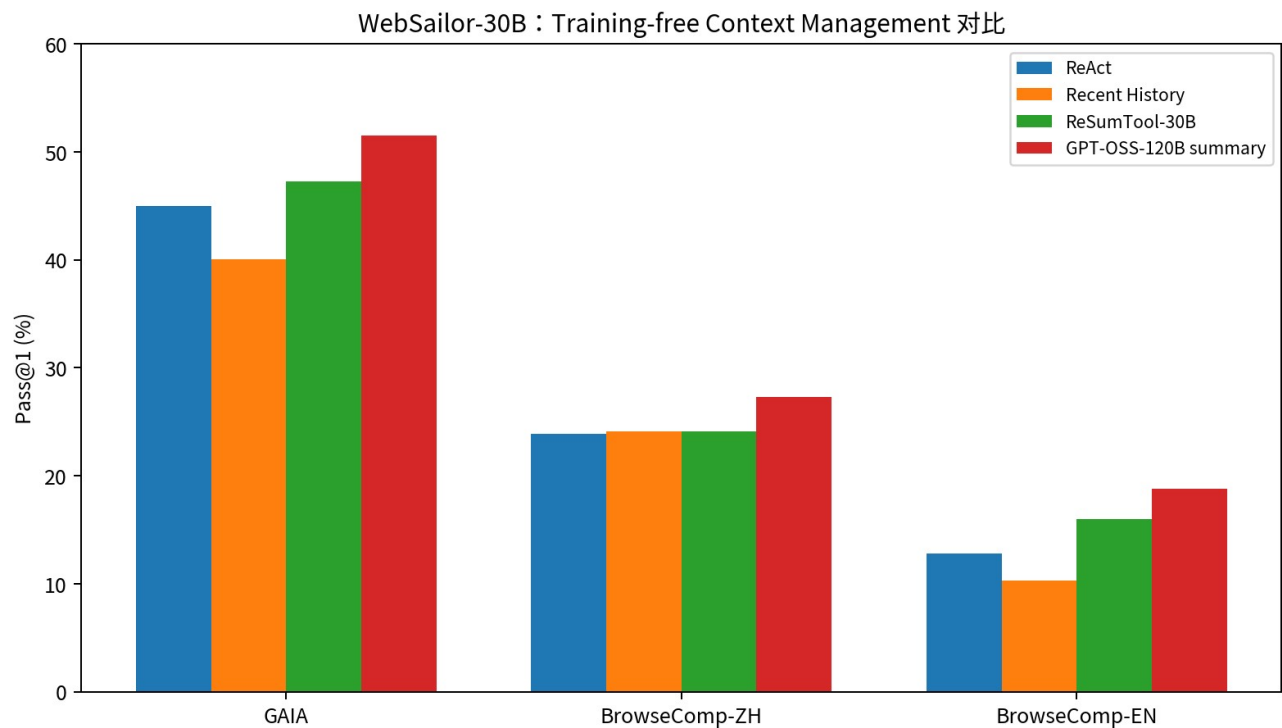


图 5 | WebSailor-30B: Training-free Context Management

5.1 WebSailor-30B 关键结果

Paradigm / Summary Tool	GAIA P@1	BrowseComp-ZH P@1	BrowseComp-EN P@1
ReAct	45.0	23.9	12.8
Recent History	40.1	24.1	10.3

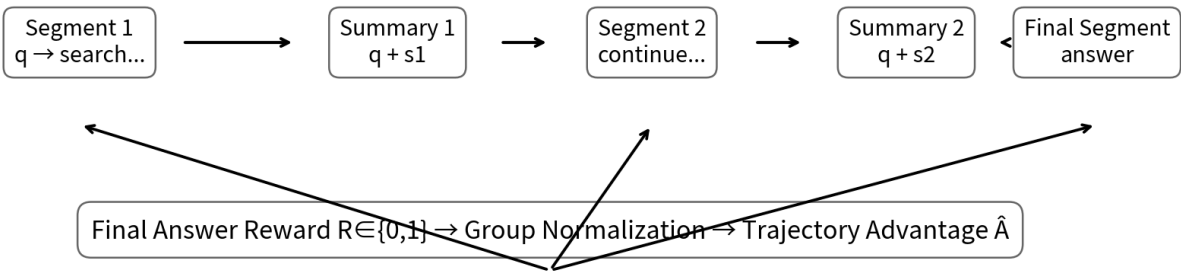
Paradigm / Summary Tool	GAIA P@1	BrowseComp-ZH P@1	BrowseComp-EN P@1
MEM1	33.3	25.0	12.7
ReSum + ReSumTool-30B	47.3	24.1	16.0
ReSum + GPT-OSS-120B	51.5	27.3	18.8
ReSum + Qwen3-235B	46.9	25.7	17.2
ReSum + DeepSeek-R1-671B	49.2	27.1	13.7

最值得注意的不是某个单点最高值，而是三个规律：一，简单截断对部分任务反而退化；二，ReSum 与现有 ReAct Agent 的兼容性明显更好；三，Summary Tool 质量对最终 Agent 成功率是一级变量，且专项 ReSumTool-30B 能以较小规模达到接近更大模型的效果。[S1]

Know-How Context compression 组件应该作为独立模型做 eval；不要默认“任意便宜模型摘要一下都差不多”。

6. ReSum-GRPO：让 Policy 真正学会“从摘要继续思考”

Training-free ReSum 虽然有效，但原 Agent 的训练分布是“从完整 history 继续 reasoning”，而不是“从 question + summary 恢复”。ReSum-GRPO 的目标是让 Policy 对这种 compressed-state query 形成真正的适配。[S1]



Advantage Broadcasting：同一 rollout 的所有 segments 共享最终成败的 advantage；Observation 不参与 loss。

图 6 | ReSum-GRPO：Segmented Trajectory + Advantage Broadcasting

6.1 Trajectory Segmentation

每发生一次 summarization，就把完整 rollout 切成新的训练 episode。第 1 段从原始 q 开始，第 k 段从 q + summary(k-1) 开始。Tool observations 属于环境返回，在 loss 中 mask 掉。没有发生 summary 的短轨迹仍按标准 GRPO 处理。[S1]

6.2 为什么广播 Advantage，而不是给每段单独设计 Reward

- 早期 segment 的价值通常只有在最后解题成功时才显现，难以局部标注。
- 最终答案 correctness 可以作为统一 trajectory outcome。
- Group 内 reward 标准化得到 advantage，再广播给同一 rollout 的所有 segments。
- 这样既奖励早期 “为好 summary 收集了有用证据”，也奖励后段 “能从 summary 恢复并继续推理”。

6.3 训练参数

项	设置
训练样本	SailorFog-QA 随机 1K
batch size	64
group size G	8
learning rate	2e-6
epochs	4
GRPO tool calls	40
ReSum-GRPO tool calls	60
ReSum segment token budget	4K prompt + 28K response
format violation	trajectory reward=0

7. RL 实验：数据只有 1K，但范式适配增益明显

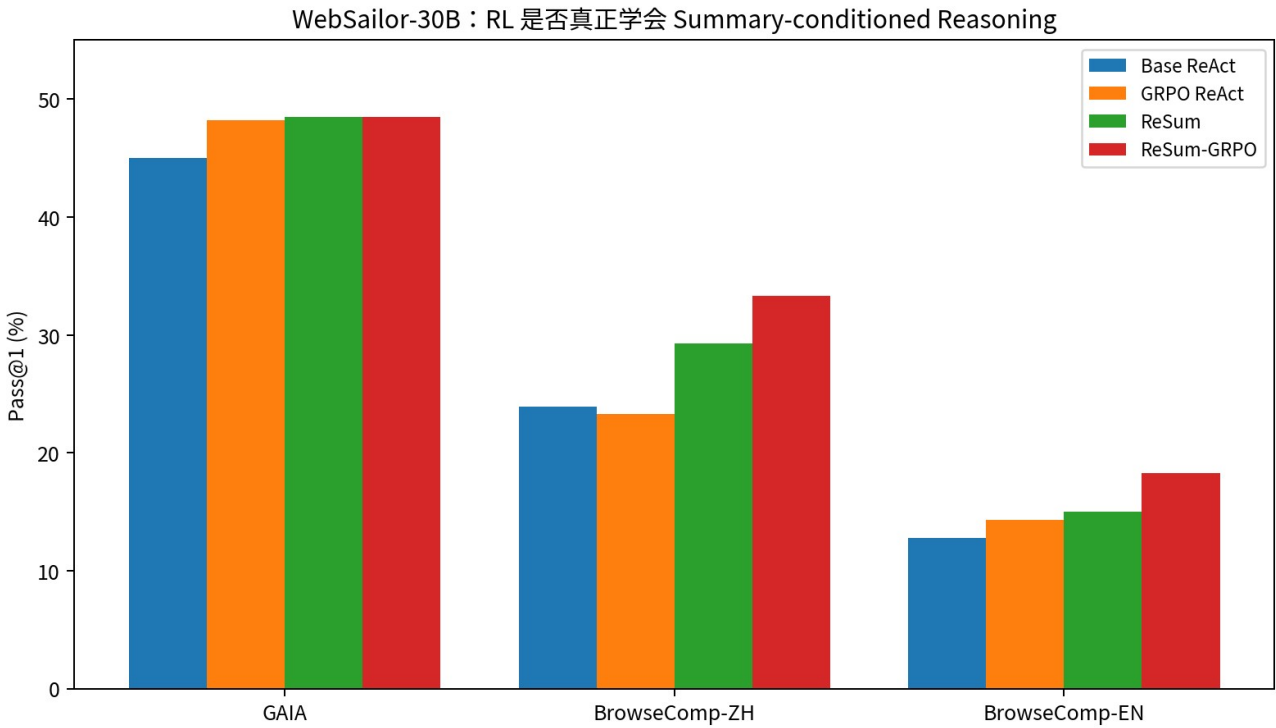


图 7 | WebSailor-30B: ReSum-GRPO 的 Pass@1

Model / Training	GAIA	BrowseComp-ZH	BrowseComp-EN
WebSailor-30B ReAct	45.0	23.9	12.8
GRPO + ReAct	48.2	23.3	14.3
ReSum (training-free)	48.5	29.3	15.0
MEM1-GRPO	35.7	29.1	19.5
ReSum-GRPO	48.5	33.3	18.3

论文总结 ReSum-GRPO 相比 ReSum training-free 可再获得最高约 8.2 个百分点的提升，并强调仅使用 1K 数据。这里应避免把结果理解为“GRPO 算法本身的胜利”：关键是训练目标与新的 Context Paradigm 对齐，即 Policy 学会了 summary-conditioned reasoning。[S1]

可迁移结论 当 Runtime 改变了 Agent 的 observation/state representation，最好给 Policy 做 paradigm adaptation；否则模型虽然“能读”，不代表它“会用”。

8. Credit Assignment：ReSum 对长程 Agent RL 的更一般启发

长程 Agent RL 的困难之一是一个最终 reward 对应几十步工具交互。ReSum 的 segment boundary 提供了一种天然的训练切分：Context compaction point 同时可以成为 episode boundary。Advantage broadcasting 则提供低成本的跨段 credit propagation。[S1]

可推广到哪些场景？

- Deep Research 的 research round / workspace reconstruction。
- Coding Agent 的 checkpoint / subtask completion。
- 多文档分析中的阶段性 evidence consolidation。
- 浏览器 Agent 的 page cluster / site-level phase。
- 长流程业务 Agent 的 state snapshot。

设计模式 State Compaction Boundary 可以同时承担三种职责：Inference memory reset、Training episode segmentation、Observability checkpoint。

9. 成本与效率：更长 horizon 不是免费的

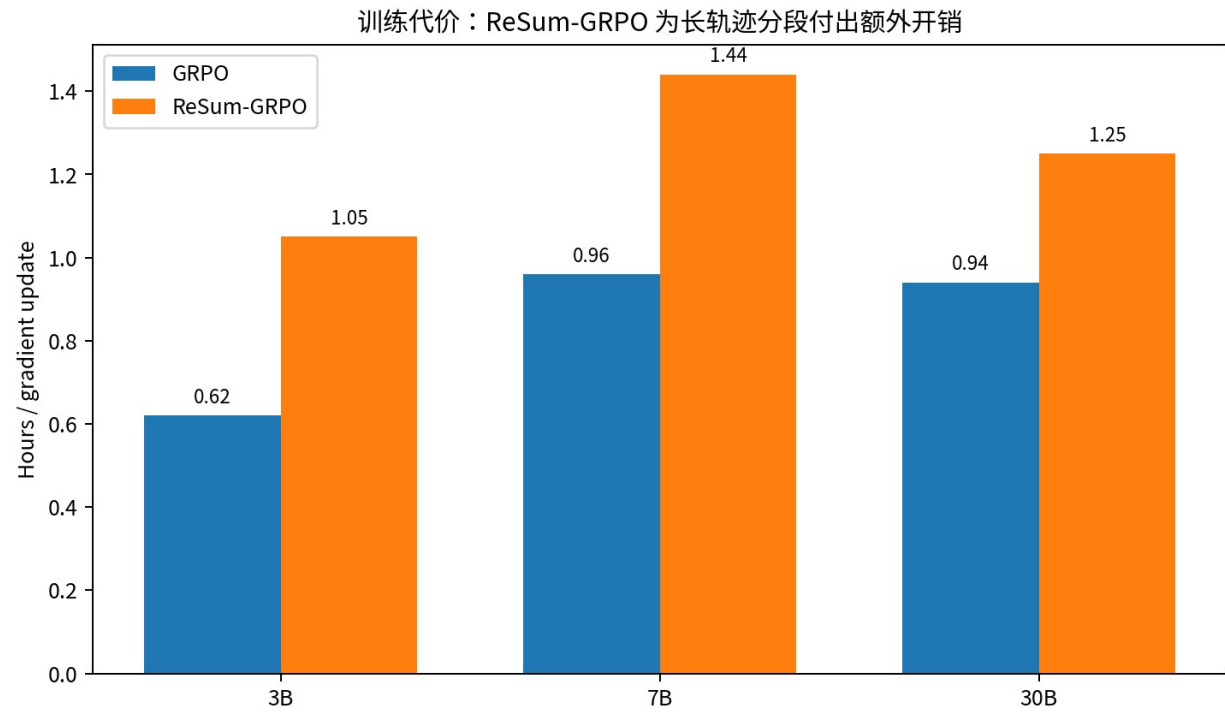


图 8 | ReSum-GRPO 单次 gradient update 的训练时间

论文报告，相比标准 GRPO，ReSum-GRPO 因为要生成摘要、继续 rollout、并更新更多 trajectory segments，单个 gradient update 的时间增加约 33%-69%。在成功样本上，ReSum-GRPO 的总 token/tool 使用通常也高于 ReAct，但论文认为总体成本仍在合理范围，典型可到原始成本约 2 倍。[S1]

9.1 为什么“更高成功率 + 更多 token”仍可能划算

- 失败的 Research Agent 往往意味着整次调用成本完全浪费。
- Context summary 把更多预算用在有效后续探索，而不是重复处理所有历史。
- 生产系统可以把 summary model 做成更便宜的 MoE / 专项模型，降低 compaction 边际成本。
- 更重要的是，预算现在可控：tool-call limit、summary watermark、max segments 都能显式约束。

10. ReSum vs Recent History vs MEM1：三种 Context Philosophy

方法	工作区策略	优点	缺点	适合场景
Recent History	只保留最近 N tokens	最简单、零额外模型	关键旧证据不可逆丢失	低价值对话/临时应急
MEM1	每轮重写 memory state	上下文近似恒定	对 Policy 指令跟随要求高；范式变化大	原生训练的 memory agent
ReSum	接近上限时周期性全局压缩	兼容 ReAct；压缩频率低	summary 是信息瓶颈；需好 summarizer	改造已有 Search Agent
Full ReAct	保留所有历史	信息保真最高	context / cost 随 horizon 增长	短程工具任务

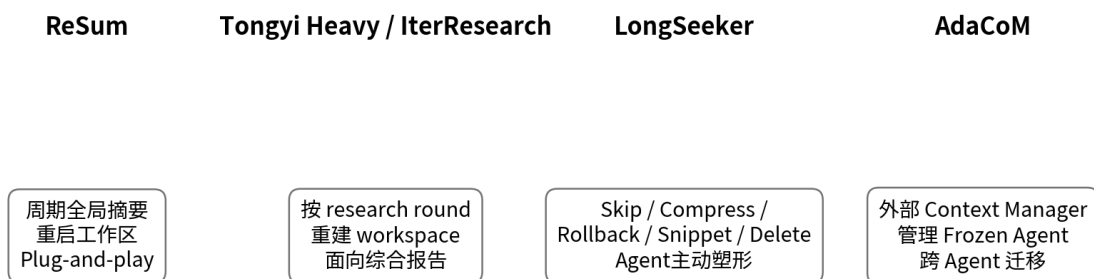
论文还指出，MEM1 的不断 memory consolidation 会产生显著 token overhead；ReSum 由于 summary 是稀疏触发，在性能-成本权衡上更容易部署。[S1]

11. 与 Tongyi DeepResearch Heavy / IterResearch 的关系

ReSum 和 Tongyi DeepResearch Heavy Mode 共享同一个根本思想：不要把一个越来越长的研究过程永久保存在单一工作区。ReSum 更偏“通用 Context Compaction primitive”，压缩整个 interaction history 并从 question+summary 重启；Heavy/IterResearch 更偏“research round orchestration”，每轮重建 focused workspace，并维护中心报告/阶段产物。两者可以组合：ReSum 管单个 Agent 的长程工作内存，IterResearch 管跨 research rounds 的 artifact state。

组合式架构 Agent 内：ReSum / compaction；Agent 外：Evidence Store / Report State / Research Rounds。不要用同一个 summary 同时承担所有层级的 memory。

12. 2026 后视角：ReSum 之后，Context Management 正在变成独立研究方向



2026 方向：Context Management 正从“固定摘要策略”走向“可学习的 Context Action Space + 独立 Context Policy”。

图 9 | 从固定 Summary 到 Elastic / External Context Policy

2026 年的 LongSeeker 提出 Context-ReAct，把 Context Management 变成 Skip、Compress、Rollback、Snippet、Delete 五种原子动作，并让 Agent 在 reasoning loop 中主动塑形工作区；论文报告在 BrowseComp / BrowseComp-ZH 上显著超过若干 2025 基线。AdaCoM 则走另一条路线：冻结目标 Agent，训练外部 Context Manager 用 RL 修改其上下文，强调不同能力 Agent 需要不同压缩强度。[S4][S5]

ReSum 在演上的位置

- 它证明了“Runtime 级 Context State Representation”本身就是能力杠杆。
- 它证明 summary-conditioned reasoning 可以通过 RL 学习，而不必修改模型架构。
- 它把 compaction boundary 与 long-horizon credit assignment 连接起来。

- 它也暴露了固定全局摘要的局限：摘要要是 lossy bottleneck，无法像更细粒度 Context Actions 那样选择性保真。

13. 最值得沉淀的 Know-Why

1. Agent 不是 Chat Log Processor

真正长程 Agent 需要显式 state management。完整日志用于审计，不应等同于每轮推理输入。

2. “更多上下文”与“更好上下文”是两件事

Context size 解决容量，Context policy 解决选择、组织、去噪与恢复。

3. Summary 是 State Representation

摘要的评价指标应该是“能否支持后续决策”，而不是 ROUGE、长度或语言流畅度。

4. Runtime 与 Policy 需要协同训练

只换 Context 结构而不做适配，模型可能会错误使用 summary；ReSum-GRPO 的增益证明了这一点。

5. Compression Boundary 是系统级 primitive

它同时影响 inference、training、observability、cost control 和 failure recovery。

6. 是可系的合理起点

Agent 自主管理上下文听起来更智能，但没有稳定 eval 前，deterministic budget gate 更容易生产化。

14. 最值得沉淀的 Know-How：如何做一个 ReSum-style Context Manager

1. 把 Full Trajectory Log 与 Working Context 分离存储。
2. 在每一步记录 context tokens、tool calls、evidence count、repeat-query rate、remaining budget。
3. 设置 summary trigger：例如 75%-90% context watermark + tool-output spike + max turns。
4. Summary 输出结构化 state：verified facts / candidates / rejected / unresolved / sources / hypothesis。
5. 压缩后真正 reset 工作区，只注入 system + original task + compacted state。
6. 上一版 summary 可以作为下一次 summarizer 输入，但要防止错误累积；保留 provenance refs 以便回溯原始 evidence。
7. 对 summary model 建独立 eval：resume success、fact retention、source retention、false insertion、next-step utility。
8. 训练 Policy 时把每个 compaction segment 当 episode；tool responses mask loss。
9. 用最终 outcome 做 trajectory reward；若采用 GRPO/PPO，可把 advantage 传播给所有 segments。
10. 线上记录 termination taxonomy：answer / token limit / call budget / format error / summary failure / tool failure。

MVP 优先级 先做 Runtime Compaction + Summary Eval，再考虑 Context Policy RL。一个稳定的 summary/reset loop 往往比复杂 Memory 架构更快产生收益。

15. Production Architecture：不要只有 Summary

层	职责	建议持久化内容
Policy Layer	Reason / Search / Visit / Answer	当前 working context
Compaction Layer	Trigger + Summary + Reset	summary snapshots + metrics
Evidence Store	保存可验证原始证据	URL / passage / source / timestamp
Artifact State	研究报告/表格/结论等长期产物	section drafts / claims / citations
Audit Log	完整 trajectory 与工具返回	append-only event log
Budget Controller	token/tool/time/cost 限制	remaining budget / stop reason
Verifier	检查 summary 和 final answer	fact retention / citation / contradiction

如果只保留 summary 而丢弃原始 evidence，系统会把一次有损压缩变成不可逆事实来源。生产级设计应该是：summary 负责工作记忆，Evidence Store 负责事实底座，Audit Log 负责可复现性。

最终抽象 Long-Horizon Deep Research = Policy × Context Manager × Evidence Store × Artifact State × Budget Controller × Verifier

16. 局限与风险

- Summary hallucination / omission：一旦关键事实被压缩掉，后续 Policy 很难知道自己丢了什么。
- Error accumulation：递归 summary-of-summary 可能逐轮漂移；应保留 evidence IDs 并周期性从原始 evidence 重建。
- Trigger 粗粒度：只按 token watermark 不能区分“高价值长历史”与“低价值噪声”。
- Outcome reward 稀疏：Advantage broadcasting 解决跨 segment 传播，但仍不能精准区分是哪次 summary 造成成败。
- 论文训练主要在短答案 web search benchmark；对长篇 research report、代码 Agent、浏览器 Agent 的适配仍需独立验证。
- LLM-as-Judge 虽经一致性验证，但仍是实验选择；生产系统应同时保留 benchmark-specific deterministic checks。

17. 研究结论

ReSum 最重要的贡献不是一个新的“摘要 Prompt”，而是重新定义了 Agent 历史的生命周期：历史可以被编译、替换、重启，而完整日志则留在 Runtime 外部。这一变化把 Context Engineering 从静态 prompt selection 推进到动态 state transition。

如果把 2024-2026 Deep Research 的演进串起来，可以看到一条清晰主线：早期系统优化 Search / Planning；随后系统开始训练 Agent Policy；再往后，真正限制 horizon 的问题转向 Context State、Evidence State 和 Verifier。ReSum 正处在这个转折点上。

一句话结论 不要让长程 Agent 永远背着完整过去前进；让它携带一个经过验证、可恢复、面向目标的状态继续工作。

18. 主要来源与证据边界

[S1] Wu et al. “ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization” , arXiv:2509.13313v3, 2026-03-26。本文方法、公式、实验、训练参数、成本分析的主要来源。

[S2] Alibaba-NLP/DeepResearch · WebAgent/WebResummer/README.md。开源 quick start、ReSumTool / ReSum-GRPO 项目说明与公开实现位置。

[S3] Alibaba-NLP/DeepResearch · WebAgent/WebResummer/src/react_agent.py 与 summary_utils.py。Runtime trigger、reset、summary server、token/call budget 等源码事实。

[S4] Lu et al. “LongSeeker: Elastic Context Orchestration for Long-Horizon Search Agents” , arXiv:2605.05191, 2026。用于 2026 后续 Context-ReAct / elastic context 对比。

[S5] Yi et al. “Learning Agent-Compatible Context Management for Long-Horizon Tasks” , arXiv:2605.30785, 2026。用于外部 Adaptive Context Manager 方向对比。

证据边界：ReSum 论文的主实验集中于 GAIA / BrowseComp / BrowseComp-ZH 的 web search short-answer evaluation；本报告关于“生产级 Evidence Store、Artifact State、Verifier”的部分属于基于论文与现有 Agent 工程模式的工程推导，不应被误读为 ReSum 原论文已实现的组件。